Librería · Tercer Prototipo

Guía del proyecto para el equipo: qué hace cada parte, cómo lo hace y cómo cumple cada punto de la rúbrica.

Curso	Desarrollo de Aplicaciones Empresariales · Universidad de Ibagué · 2026B
Profesor	Carlos Lugo
Integrantes	Misael Gallo Tangarife · Sara Zambrano · Santiago Guimel · Alejandra González
Versión	3.0.0
Repositorio	**https://github.com/Milan32555/tercer-prototipo-libreria**
Caso	Libro (maestro, colección A) → Ejemplar (detalle, colección B)

Contenido

1. El proyecto en una página
2. Cómo levantar la demo
3. Requisito 1 · Arquitectura
4. Requisito 2 · GraphQL, dos clientes y MongoDB
5. El modelo: atributos, validaciones y colecciones
6. Insertar (15 %)
7. Eliminar (20 %)
8. Listar con parámetro (25 %)
9. Actualizar (20 %)
10. Consulta con parámetros (20 %)
11. Demás puntos del enunciado
12. Lo que pidió el profesor en clase
13. Patrones de diseño
14. Pruebas y resultados
15. Guion para presentar en clase
16. Preguntas probables y respuestas
17. Mapa de archivos

1. El proyecto en una página

Es una aplicación distribuida para gestionar el inventario de una librería con dos objetos **maestro → detalle**: un **Libro** (un título) tiene 0..* **Ejemplares** (las copias físicas). Un servidor central en Java expone todas las operaciones con **servicios web GraphQL**; dos clientes en otros lenguajes (una **web en React/TypeScript** y una **app Android en Flutter/Dart**) las consumen. La información se guarda en **MongoDB** (base no relacional) usando el framework **Spring Data MongoDB**, en dos colecciones: `libros` y `ejemplares`.

Cliente web
React · TypeScript
Vite · puerto 5173

→

App móvil
Flutter · Dart
Android (celular por USB)

→

Servidor GraphQL
Java 17 · Spring Boot 4.1.2
puerto 8081 · `/graphql`

→

MongoDB
Spring Data MongoDB
colecciones `libros` y `ejemplares`

Cómo se cumple la rúbrica (resumen)

Punto de la rúbrica	Peso	Cómo lo cumplimos (detalle en cada sección)
REQ Arquitectura	Requisito	Paquetes separados: `model`, `dto`, `repository`, `servicio`, `controller`, `config`, `excepciones`, `tareas`. Las vistas (GUI) están en los clientes y solo hacen interfaz.
REQ Implementación de PD	Requisito	GraphQL (Spring for GraphQL); clientes en TypeScript y Dart (distintos a Java); MongoDB con el framework Spring Data MongoDB; patrones MVC, Builder, Observer y Singleton.
Insertar	15 %	Adicionar Libro y Adicionar Ejemplar, con todos los atributos, validación en el cliente y en el servidor.
Eliminar	20 %	Se busca primero, se muestra TODA la información (incluidos opcionales, `_id`, `libroId` y versión) y se confirma en un diálogo.
Listar con parámetro	25 %	Grilla paginada; filtros; dos consultas propias (`librosPorPrecioYGenero` y `ejemplaresConLibro`); Listar Ejemplares muestra todos sus atributos + título y autor del libro.
Actualizar	20 %	Se busca primero, se muestra TODA la información y se edita; el servidor detecta si otro usuario lo cambió (versión).
Consulta con parámetros	20 %	UN objeto por un parámetro, con dos criterios por objeto: Libro por ISBN o título; Ejemplar por código o ISBN + ubicación.

> **Regla de oro de la rúbrica:** cada caso de uso se califica **en las dos tecnologías** (web y app). Todo lo que se muestra en esta guía funciona igual en los dos clientes, porque los dos llaman a las mismas operaciones GraphQL.

Números que conviene saber

Qué	Valor
Datos de prueba al arrancar con la base vacía	12 libros y 20 ejemplares (10 disponibles, 6 prestados, 2 dañados, 2 dados de baja)
Pruebas automáticas	Servidor 74 · Web 84 + 32 E2E (escritorio y celular) · App 46 + integración + recorrido en el celular real
Carga (1.000 libros y 5.000 ejemplares)	Listar paginado ≈ 7 ms; consulta `$lookup` ≈ 6–9 ms

2. Cómo levantar la demo

Opción rápida (la que usamos para presentar)

1. Conectar el celular por USB con la **depuración USB** activada (aceptar "Permitir" en el celular si lo pregunta).
2. En el escritorio, abrir la carpeta **"Tercer Prototipo - Demo Libreria"** y dar doble clic a **INICIAR DEMO (doble clic).cmd**.
3. El script hace todo, en orden:
   - **MongoDB:** si no está escuchando en `localhost:27017`, intenta el servicio de Windows y, si pide administrador, lo arranca como proceso del usuario.
   - **Servidor:** abre la ventana "Servidor GraphQL - puerto 8081" (ahí se ven los logs y qué cliente hizo cada operación).
   - **Web:** abre la ventana "Cliente web React" y el navegador en `http://localhost:5173`.
   - **App:** hace `adb reverse tcp:8081 tcp:8081` (el celular llega al servidor del PC por el cable), instala el APK y abre la app "Librería".
4. Para apagar: cerrar las ventanas "Servidor GraphQL" y "Cliente web React".

En la misma carpeta está **Como probar.txt** con los ISBN y códigos de ejemplo.

Opción manual (por partes)

```
# 1. Servidor (si la base está vacía carga los 12 libros y 20 ejemplares)
cd servidor-libreria-mongo
mvnw spring-boot:run

# 2. Cliente web
cd cliente-web-react
npm install
npm run dev                      # http://localhost:5173

# 3. App en el celular
adb reverse tcp:8081 tcp:8081
cd cliente-movil-flutter
flutter run --dart-define=URL_GRAPHQL=http://localhost:8081/graphql
```

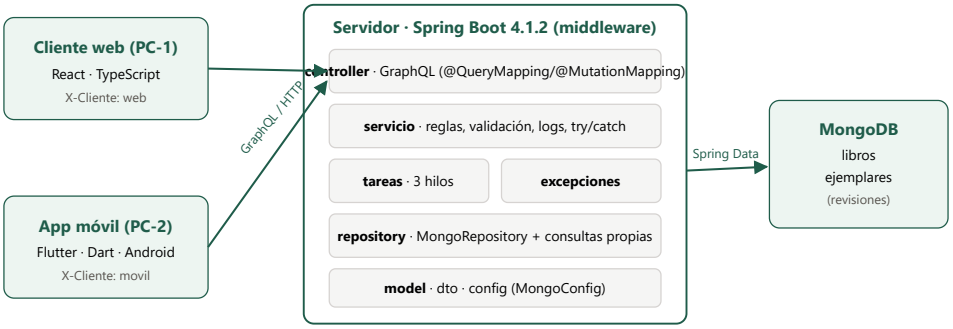
Datos de ejemplo para probar

Para	Usar
Libro con ejemplares (no se deja eliminar)	`978-0307474728` · Cien años de soledad (3 ejemplares en "Estante A1")
Libro sin ejemplares (sí se elimina)	`978-8420401751` · Delirio
Consultar por título sin tildes	`maria` → María · `la voragine` → La vorágine
Ejemplar dañado	`EJ-007` (Bodega, observación "Lomo despegado")
Ejemplar disponible para prestar (+1 préstamo)	`EJ-003`, `EJ-006`, `EJ-009`
Filtro de precio con resultados	de 40.000 a 60.000
Consola GraphiQL para mostrar el esquema	`http://localhost:8081/graphiql`

> **Si algo falla:** "No se pudo conectar con el servidor" → la ventana del servidor no está abierta o MongoDB no arrancó. En el celular: revisar que el cable siga conectado y volver a ejecutar el script (repite el `adb reverse`).

3. Requisito 1 · Arquitectura

> *"La aplicación debe cumplir con las especificaciones de diseño, las clases GUI (vistas), Model y estructurales deben existir así como la clase XXX en sus respectivos paquetes. Las operaciones deben ser coherentes con la naturaleza de la clase, por ej.: la clase GUI debe realizar procesos asociados a interfaz gráfica."* — ***Si no se cumple, la nota es 0.***



Arquitectura web distribuida (como la figura 4 del enunciado, con base NO relacional).

Paquetes del servidor (`com.libreria.servidor`) y qué hace cada uno

Paquete	Clases	Responsabilidad (coherente con su naturaleza)
`model`	`Libro`, `Ejemplar`, `Genero`, `EstadoEjemplar`, `EjemplarConLibro`, `Pagina`, `DetalleLibro`, `ResumenInventario` …	Clases estructurales: los documentos de MongoDB y los tipos de respuesta. No tienen lógica.
`dto`	`LibroInput`, `EjemplarInput`, `@IsbnValido`	Lo que llega del cliente, con sus validaciones (Bean Validation).
`repository`	`LibroRepository`, `EjemplarRepository` + …`CustomImpl`	Acceso a MongoDB: consultas derivadas y consultas propias (Criteria y agregaciones).
`servicio`	`ServicioLibro`, `ServicioEjemplar`, `ServicioRevisiones`, `ObservadorColecciones`, `GeneradorDatosCarga`	Lógica de negocio (el "Model" de MVC): reglas, validación, logs y manejo de errores.
`controller`	`LibroController`, `EjemplarController`, `ResumenController`	Traducen entre GraphQL y los servicios. No tienen reglas de negocio.
`config`	`MongoConfig`, `EjecutorTareasConfig`, `LocalDateTimeScalarConfig`, `RegistroOperaciones`, `DatosIniciales`	Conexión a Mongo, hilos, escalar de fechas, registro por cliente y datos de prueba.
`excepciones`	`LibreriaException`, `ManejadorErrores`	Errores de negocio y su traducción a mensajes GraphQL en español.
`tareas`	`ServicioResumenInventario`	Servicio con 3 tareas concurrentes.

Clientes (las clases GUI)

	Web · `cliente-web-react/src`	App · `cliente-movil-flutter/lib`
Vistas: una por caso de uso, solo interfaz (las reglas las decide el servidor)	`paginas/libro/*`, `paginas/ejemplar/*`	`pantallas/libro/*`, `pantallas/ejemplar/*`
Acceso a datos	`graphql/` (`clienteGraphQL.ts` Singleton)	`servicios/` (`cliente_graphql.dart` Singleton)
Modelo · Observer	`types.ts` · `observer/`	`modelos/` · `observador/`

4. Requisito 2 · GraphQL, dos clientes y MongoDB

> *"Debe implementar la tecnología de Servicios Web GraphQL. Los clientes deben estar desarrollados en un lenguaje de programación diferente al del servidor. Cada caso de uso se califica de acuerdo a su funcionalidad en LAS DOS tecnologías. La aplicación debe almacenar la información en una base de datos NO relacional y utilizar un framework." —* ***Requisito: si no se cumple, nota 0.***

4.1 GraphQL

El contrato está en `servidor-libreria-mongo/src/main/resources/graphql/schema.graphqls`. Spring for GraphQL conecta cada operación con un método del controller por su nombre (`@QueryMapping` para consultas y `@MutationMapping` para cambios). Los clientes envían un `POST /graphql` con la consulta y las variables, más el encabezado `X-Cliente` (web o movil).

Operación	Para qué caso de uso
`crearLibro` · `crearEjemplar`	Insertar
`eliminarLibro(isbn)` · `eliminarEjemplar(codigo)`	Eliminar
`actualizarLibro(input)` · `actualizarEjemplar(input)`	Actualizar (un solo input para crear y actualizar)
`libroPorIsbn` · `libroPorTitulo`	Consultar Libro (2 criterios)
`ejemplarPorCodigo` · `ejemplaresPorLibroYUbicacion`	Consultar Ejemplar (2 criterios)
`libros` · `librosPorPrecioYGenero`	Listar Libros: todos / por filtro (consulta propia)
`ejemplares` · `ejemplaresConLibro`	Listar Ejemplares: todos / con su libro y filtros (consulta propia)
`detalleLibro(isbn)`	Consulta propia maestro + detalle
`resumenInventario` · `revisionLibros` · `revisionEjemplares`	Inicio (3 tareas) y Observer entre clientes

4.2 Dos clientes en lenguajes diferentes al servidor

Servidor en **Java 17**; cliente web en **TypeScript** (React, llama con `fetch`); app en **Dart** (Flutter, llama con el paquete `http`). Los dos usan las mismas operaciones GraphQL.

4.3 Base NO relacional con un framework

- **MongoDB 8**, base `libreria`, colecciones **`libros`** (A) y **`ejemplares`** (B). La colección `revisiones` es solo un contador técnico del Observer.
- **Framework: Spring Data MongoDB.** Las clases tienen `@Document("libros")` / `@Document("ejemplares")`, `@Id`, `@Indexed`, `@Version`.
- **Conexión:** `config/MongoConfig.java` extiende `AbstractMongoClientConfiguration` (como el POC del profesor); la URL y la base están en `application.properties`.
- **Repositorios:** `LibroRepository extends MongoRepository<Libro, String>`. Spring Data genera `findByIsbn`, `existsByIsbn`, `countByLibroId`… a partir del nombre del método.
- **Prohibido usar listas:** no existe ninguna `List` que guarde datos. Se borraron los repositorios Singleton en memoria del segundo proyecto; todo lee y escribe en MongoDB.

5. El modelo: atributos, validaciones y colecciones

El enunciado pide llave primaria, obligatorios y opcionales, validaciones y al menos 5 atributos (**int, double, String, LocalDateTime y otro**). Las validaciones están en el servidor (Bean Validation, `@Valid`) y se repiten en los clientes para avisar antes de enviar.

Libro · colección `libros` (A, maestro)

Atributo	Tipo	Oblig.	Validación / nota
`id`	String (`ObjectId`)	PK	`@Id`: el `_id` que asigna MongoDB
`isbn`	String	Sí	Único (índice único); ISBN-10/13 con dígitos y guiones (`@IsbnValido`)
`titulo`	String	Sí	No vacío, máx. 200
`autor`	String	Sí	No vacío, máx. 120 · índice
`precio`	**double**	Sí	> 0 (`@Positive`) · índice
`numeroPaginas`	**int**	Sí	> 0
`fechaPublicacion`	**LocalDateTime**	Sí	No futura (`@PastOrPresent`)
`genero`	**enum Genero** (el "otro")	Sí	FICCION, NO_FICCION, CIENCIA, HISTORIA, INFANTIL, POESIA, OTRO · índice
`editorial`	String	**No**	Máx. 120
`sinopsis`	String	**No**	Máx. 500
`version`	Long	—	`@Version`: concurrencia optimista

Ejemplar · colección `ejemplares` (B, detalle)

Atributo	Tipo	Oblig.	Validación / nota
`id`	String (`ObjectId`)	PK	`@Id`
`codigo`	String	Sí	Único; formato `EJ-000` (EJ- y 3 a 6 dígitos)
`libroId`	`ObjectId`	Sí	**Llave foránea** → `libros._id`; el libro debe existir · índice
`estado`	enum EstadoEjemplar	Sí	DISPONIBLE, PRESTADO, DANIADO, DADO_DE_BAJA · índice
`ubicacion`	String	Sí	No vacía, máx. 60
`fechaIngreso`	LocalDateTime	Sí	No futura
`precioReposicion`	double	Sí	> 0
`numeroPrestamos`	int	Sí	≥ 0; el servidor suma 1 al pasar a PRESTADO
`observaciones`	String	**No**	Máx. 300
`version`	Long	—	`@Version`

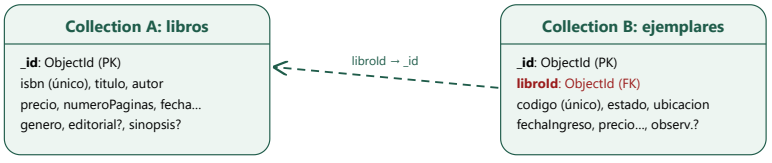


Diagrama NO relacional (como la figura 2 del enunciado). Un libro 0..1 — 0..* ejemplares.

Índices: `libros`: isbn (único), autor, precio, genero y compuesto (genero, precio) · `ejemplares`: codigo (único), libroId, estado y compuesto (libroId, ubicacion). Los clientes nunca ven el `ObjectId` como dato de trabajo: envían el **ISBN** y el servidor lo traduce a `libroId`.

6. Insertar 15 %

> *"Operación que permite adicionar un nuevo objeto de la clase XXX (ambas) de acuerdo a sus características (atributos)."*

Qué hace

Dos pantallas, una por objeto: **Libros → Adicionar** y **Ejemplares → Adicionar**. El formulario pide todos los atributos, separados en obligatorios y opcionales. Al adicionar un ejemplar, el libro se elige por su **ISBN** y debajo aparece su título y autor (o "No hay ningún libro con ese ISBN").

Cómo lo hace (flujo)

1. Pantalla
Valida el formato: obligatorios, números positivos, ISBN, EJ-000, fecha no futura.

→

2. Mutación
`crearLibro(input)` / `crearEjemplar(input)`

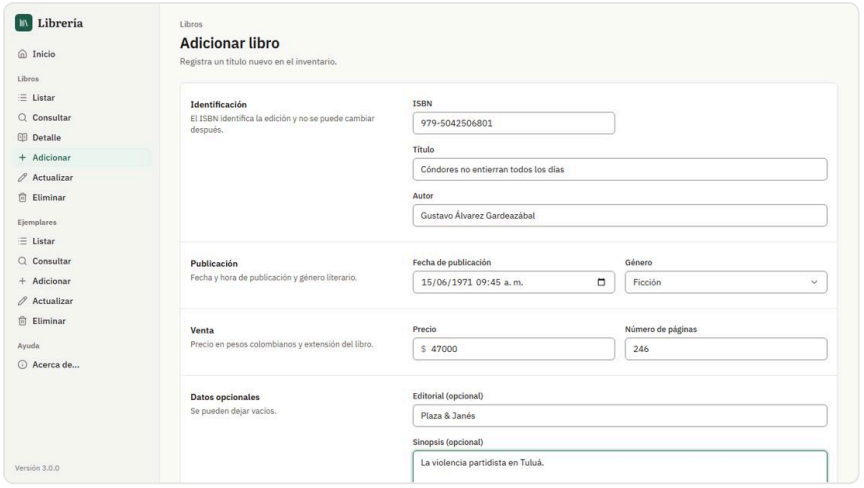
→

3. Servicio
`@Valid` (Bean Validation) y reglas: ISBN/código único; el libro debe existir.

→

4. MongoDB
`insert` con `Libro.builder()`; el índice único evita duplicados.

- **Reglas:** ISBN y código únicos; un ejemplar solo para un libro que exista (el servidor guarda su `libroId` como ObjectId).
- **Mensajes:** "Ya existe un libro con el ISBN …", "No existe un libro con el ISBN …; el ejemplar debe pertenecer a un libro registrado", y cada validación con su texto ("El precio debe ser mayor que cero.").
- **Después:** avisa el éxito y notifica al Observer, así la pantalla Listar se refresca sola.
- **Archivos:** `ServicioLibro.crear`, `ServicioEjemplar.crear`, `AgregarLibro.tsx`, `agregar_libro.dart`, `AgregarEjemplar.tsx`, `agregar_ejemplar.dart`.



Web: adicionar libro (obligatorios y opcionales)



App: adicionar libro



App: ejemplar adicionado

7. Eliminar 20 %

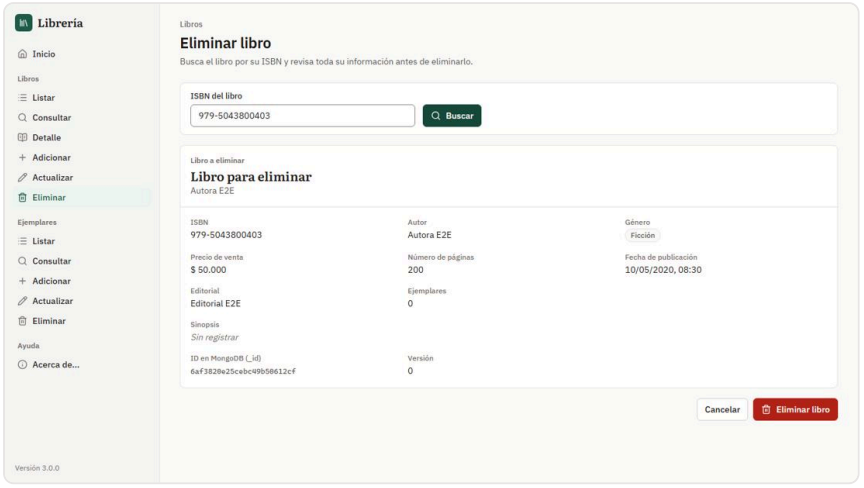
> *"Operación que permite eliminar un objeto previamente **buscado** por el usuario. Antes de ser eliminado, se debe mostrar al usuario **TODA** la información del objeto."*

Qué hace

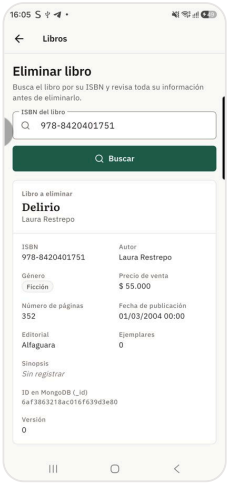
1. El usuario escribe el ISBN (libro) o el código (ejemplar) y pulsa **Buscar**.
2. Se muestra la **ficha completa**: todos los atributos, incluidos los opcionales ("Sin registrar" si están vacíos), el `_id` de MongoDB, la llave foránea `libroId`, la versión, y el título y autor del libro en el caso del ejemplar.
3. El botón **Eliminar** abre una confirmación (diálogo en la web, hoja inferior en la app). **Cancelar** no borra nada.
4. Al confirmar, se llama `eliminarLibro(isbn)` / `eliminarEjemplar(codigo)` y se avisa el resultado.

Cómo lo hace

- **Regla de negocio:** no se elimina un libro que tiene ejemplares. El servidor cuenta en Mongo (`countByLibroId`) y responde "No se puede eliminar el libro … porque tiene N ejemplar(es) registrado(s). Elimine primero sus ejemplares."
- Si otro usuario lo modificó entre la búsqueda y el borrado, MongoDB lo detecta por la versión y responde con un conflicto.
- **Archivos:** `ServicioLibro.eliminar`, `ServicioEjemplar.eliminar`, `EliminarLibro.tsx`, `eliminar_libro.dart`, `FichaLibro.tsx`, `fichas.dart`.



Web: ficha completa antes de eliminar



App: ficha completa



App: rechazo (tiene ejemplares)

8. Listar con parámetro 25 %

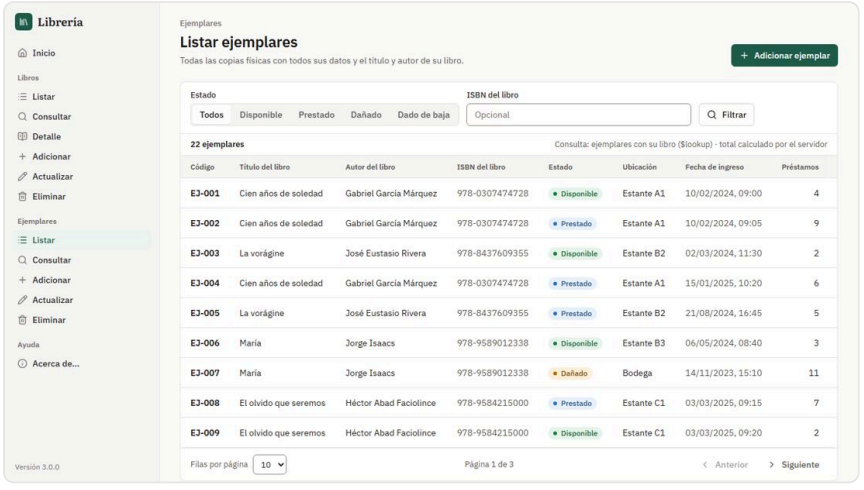
"Operación que permite Listar todos los objetos en la Aplicación a través de una grilla. Es posible filtrar la información con un parámetro. Debe haber dos consultas 'personalizadas' para listar. El listar de la Clase/Colección B debe mostrar todos sus atributos más dos atributos de la Clase/Colección A."

Exigencia	Cómo la cumplimos
Grilla con todos	Tabla con encabezado fijo en la web y en la app. Variante "todos": `libros(pagina, tamanio)` y `ejemplares(…)` . **Paginada en MongoDB** (10 por página; la web deja elegir 10, 20 o 50).
Filtro con un parámetro	**Libros:** por género y por rango de precio (mínimo y máximo). **Ejemplares:** por estado y por ISBN del libro. El filtro lo hace MongoDB, no el cliente. El total lo calcula el servidor.
Consulta propia 1	`librosPorPrecioYGenero` : en `LibroRepositoryCustomImpl` , con **Criteria** (`precio ≥ min` y `≤ max` , `genero = …`) y paginación.
Consulta propia 2	`ejemplaresConLibro` : en `EjemplarRepositoryCustomImpl` , una **agregación** `$match → $sort → $skip → $limit → $lookup → $project` que une cada ejemplar con su libro.
B con todos sus atributos + 2 de A	La grilla de Ejemplares muestra **todos** sus atributos (código, estado, ubicación, fecha, préstamos, reposición, observaciones, versión, `_id` , `libroId`) **más título y autor del libro** (y su ISBN).

La agregación del `$lookup` (lo que hace MongoDB por dentro):

```
db.ejemplares.aggregate([
  { $match:  { estado: "PRESTADO" } },                        // filtro (opcional)
  { $sort:   { codigo: 1 } }, { $skip: 0 }, { $limit: 10 },    // página
  { $lookup: { from: "libros", localField: "libroId", foreignField: "_id", as: "libro" } },
  { $unwind: "$libro" },
  { $project: { codigo: 1, estado: 1, ubicacion: 1, /* …todos… */ tituloLibro: "$libro.titulo",
autorLibro: "$libro.autor" } }
])
```

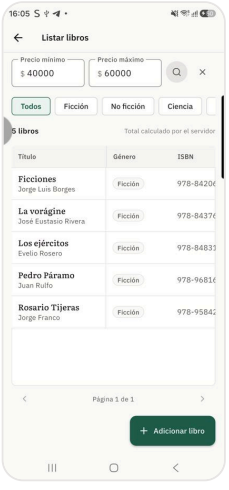
El `$lookup` se hace después de paginar: solo une los 10 ejemplares de la página, por eso es rápido (≈ 6–9 ms con 5.000 ejemplares). **Observer:** mientras la pantalla está abierta se refresca sola cuando cambian los datos (en este cliente al instante; desde otro cliente en ≤ 15 s).



Web: Listar Ejemplares (todos los atributos + título y autor)



App: Listar Ejemplares



App: libros por precio

9. Actualizar 20 %

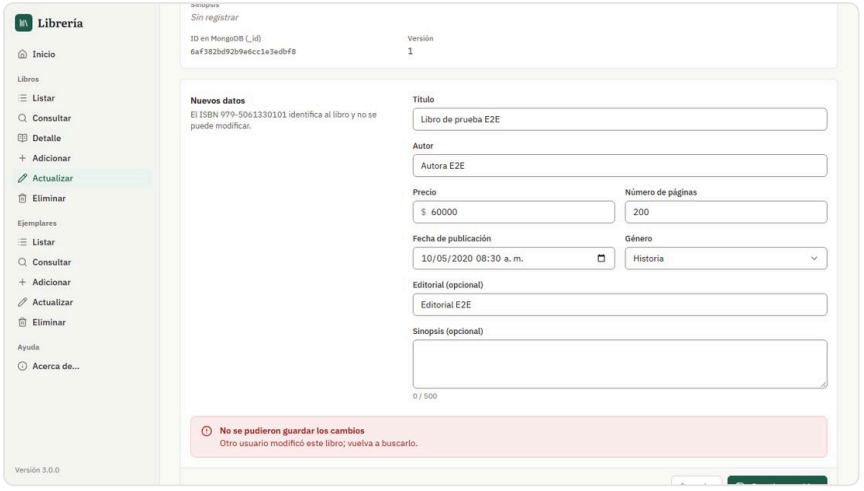
> *"Operación que permite actualizar un o varios atributos del objeto previamente **buscado** por el usuario. Antes de ser actualizado se debe mostrar al usuario TODA la información del objeto."*

Qué hace

1. Se busca por ISBN o código y se muestra la **ficha completa** ("Datos actuales").
2. Debajo, el formulario trae los valores actuales; se cambian uno o varios atributos (incluidos los opcionales).
3. Al guardar se muestra lo que respondió el servidor ("Datos guardados").

Cómo lo hace

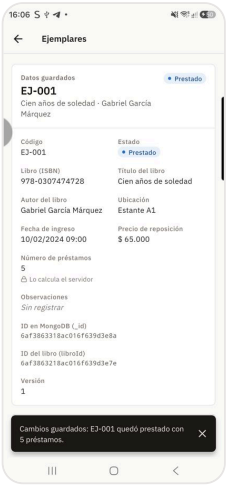
- **Un solo input** para crear y actualizar (como pidió el profesor): `LibroInput` / `EjemplarInput`. El ISBN (o el código) dentro del input dice cuál se actualiza y no se cambia.
- **Préstamos:** el número de préstamos se ve pero no se edita. Si el estado **pasa** a PRESTADO, el servidor suma 1; seguir en PRESTADO no suma.
- **Concurrencia (versión):** el cliente envía la `version` que leyó. Si otro usuario guardó antes, MongoDB no guarda y el servidor responde **"Otro usuario modificó este libro; vuelva a buscarlo."** El mensaje se muestra tal cual, junto a los botones.
- Un ejemplar no puede cambiar de libro (regla del servidor).
- **Archivos:** `ServicioLibro.actualizar`, `ServicioEjemplar.actualizar`, `ActualizarLibro.tsx`, `actualizar_ejemplar.dart`.



Web: formulario y conflicto de versión (otro usuario lo cambió)



App: datos actuales y formulario



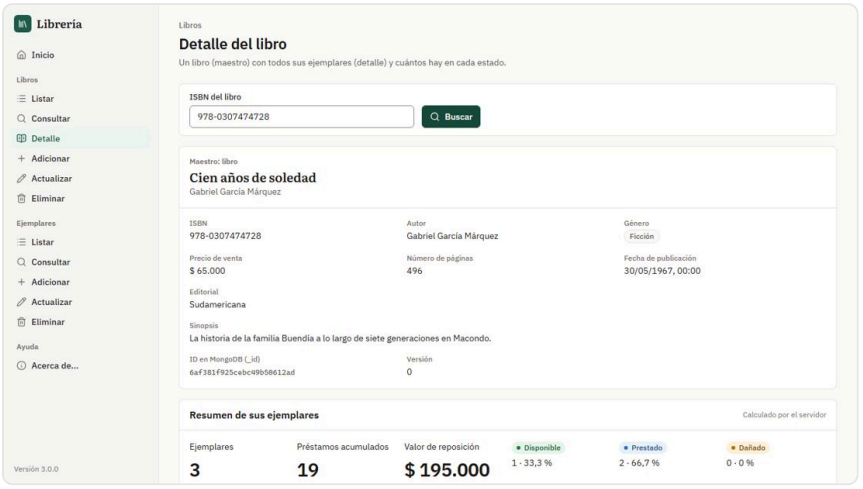
App: préstamo +1

10. Consulta con parámetros 20 %

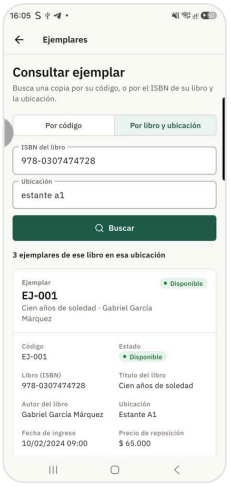
"Operación que permite consultar UN SOLO objeto mostrando sus características (atributos) a partir de un parámetro ingresado por el usuario." · Enunciado: "a partir de diferentes criterios de búsqueda (mínimo dos)".

Objeto	Criterio 1	Criterio 2
Libro	**ISBN** → `libroPorIsbn`	**Título** → `libroPorTitulo`: exacto, pero sin importar mayúsculas ni tildes ("maria" encuentra "María") gracias a una *collation* de MongoDB en español.
Ejemplar	**Código** → `ejemplarPorCodigo` (sin importar mayúsculas: "ej-007")	**ISBN del libro + ubicación** → `ejemplaresPorLibroYUbicacion`. Si hay varias copias del mismo libro en el mismo lugar las muestra todas (p. ej. Cien años de soledad en "Estante A1": EJ-001, EJ-002, EJ-004).

- Si no existe, se muestra "No hay ningún libro con el ISBN «…»" (sin error).
- Se muestra la ficha completa con todas las características.
- **Extra (consulta propia maestro + detalle): Libros → Detalle** (`detalleLibro`) muestra el libro (maestro), la grilla de sus ejemplares (detalle) y los conteos por estado, préstamos acumulados y valor de reposición. Así se cumple "una consulta debe mostrar los datos de la tabla maestro y de detalle".



Web: Detalle (maestro + detalle)



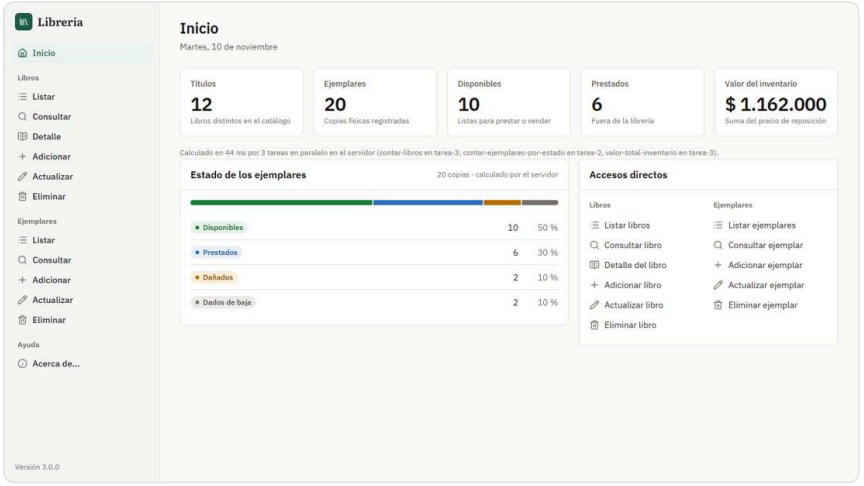
App: ejemplar por ISBN + ubicación



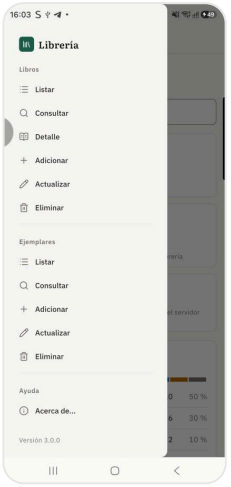
App: Detalle

11. Demás puntos del enunciado

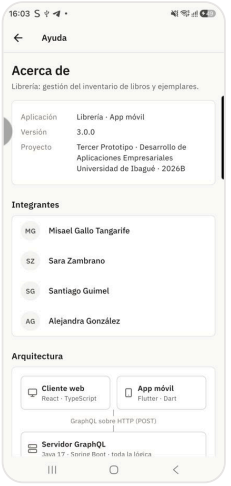
Punto del enunciado	Cómo se cumple
1. Servicios Web GraphQL	Spring for GraphQL, `schema.graphqls`, endpoint `/graphql` y consola `/graphiql`.
2. Dos clases estructurales gestionadas por colecciones NO relacionales	`Libro` y `Ejemplar` con `@Document`; colecciones `libros` y `ejemplares`.
3. Dos clientes en otro lenguaje; cada ventana un solo caso de uso	Web (TypeScript) y app (Dart). Una ruta/pantalla por caso de uso: Listar, Consultar, Detalle, Adicionar, Actualizar, Eliminar de cada objeto.
4. Consulta con mínimo dos criterios	Ver sección 10.
5. Excepciones con mensajes coherentes	`LibreriaException` con un tipo (VALIDACION, NO_ENCONTRADO, DUPLICADO, CONFLICTO, REGLA_NEGOCIO, ERROR_INTERNO). `ManejadorErrores` la convierte en un error GraphQL con mensaje en español; los clientes lo muestran tal cual. Si MongoDB no responde: "No se pudo … : la base de datos no respondió".
6. Buscar antes de eliminar/actualizar; no usar la misma ventana para más de una función	Eliminar y Actualizar empiezan con un buscador; son pantallas separadas de Consultar.
7. Menú principal + Ayuda → "Acerca de…" (integrantes y versión)	Menú lateral (web) y cajón de navegación (app), con las secciones Libros, Ejemplares y Ayuda. Acerca de muestra los 4 integrantes, la versión 3.0.0, la arquitectura y si el servidor está conectado.
8. Subir a GitHub	**https://github.com/Milan32555/tercer-prototipo-libreria** (público), con integración continua: GitHub Actions compila y prueba los tres proyectos (el servidor contra un MongoDB real). Está en verde.
9. Listar todos y filtrar por al menos un parámetro	Ver sección 8.
Interoperabilidad	Probada: un libro creado desde el celular aparece en la web; la consola del servidor dice "App móvil" o "Cliente web" en cada operación.



Web: Inicio con el menú (Libros, Ejemplares, Ayuda)



App: menú



App: Acerca de

12. Lo que pidió el profesor en clase

Pedido	Dónde y cómo
Spring Boot 4.1.2, Java 17, Maven, Jar, Properties; Web, Lombok, GraphQL, Spring Data MongoDB, Validation	`pom.xml` generado con Spring Initializr con esas opciones. Security no se incluyó (era opcional).
`@Id` en el identificador y `ObjectId` como llave foránea	`Libro.id` y `Ejemplar.id` con `@Id`; `Ejemplar.libroId` es `org.bson.types.ObjectId`. Se verificó en MongoDB que `ejemplares.libroId` es ObjectId y apunta a `libros._id`, sin huérfanos (`scripts\verificar-mongo.cmd`).
Logging en 3 niveles con try/catch	Cada servicio y controller tiene `Logger` (SLF4J). **info** = operación y resultado; **warn** = rechazo (no existe, duplicado, validación, versión); **error** = excepción inesperada en el `catch`. `logback-spring.xml`: consola con hora, nivel, hilo y clase + archivo `logs/servidor.log` rotativo.
Servicio con 3 tareas concurrentes (como el POC de Hilos y CompletableFuture)	`tareas/ServicioResumenInventario`: lanza en paralelo (1) contar libros, (2) contar ejemplares por estado con una agregación y (3) sumar el valor del inventario. Usa `CompletableFuture.supplyAsync` sobre el ejecutor propio `ejecutorTareas` (hilos `tarea-1..3`), las une con `allOf`, cada una con `orTimeout` y `exceptionally` + `logger.error`. Si una falla, las otras dos igual se entregan. Lo usa el Inicio ("Calculado en N ms por 3 tareas…").
Concurrencia	(a) Índices únicos: dos altas simultáneas del mismo ISBN → solo una se guarda (`DuplicateKeyException` → "Ya existe…"). (b) `@Version`: dos actualizaciones con la misma versión → la segunda recibe conflicto. (c) Préstamos simultáneos del mismo ejemplar → el contador no se pierde.
Cantidad de datos	Paginación en todos los Listar, filtros en MongoDB e índices. Perfil `carga`: genera 1.000 libros y 5.000 ejemplares (`mvnw spring-boot:run -Dspring-boot.run.profiles=carga`, base `libreria_carga`).
Builder	Lombok `@Builder` en `Libro` y `Ejemplar`: `Libro.builder().isbn(…).titulo(…).build()` en los servicios, la semilla y el generador.

Ejemplos reales de la consola del servidor:

```
15:00:32.883 INFO  [tarea-2] c.l.s.t.ServicioResumenInventario - Tarea contar-ejemplares-por-estado
iniciada en el hilo tarea-2
15:00:44.935 WARN  [http-nio-8081-exec-5] c.l.servidor.servicio.ServicioLibro - Crear libro rechazado: Ya
existe un libro con el ISBN 978-0307474728.
15:00:45.035 WARN  [http-nio-8081-exec-10] c.l.servidor.servicio.ServicioLibro - Conflicto de versión al
actualizar el libro 978-8420401751: otro usuario lo modificó
15:06:23.820 ERROR [tarea-2] c.l.s.t.ServicioResumenInventario - Tarea contar-libros falló:
DataAccessResourceFailureException: MongoDB no responde
16:04:10  App móvil    | mutación crearLibro (isbn=979-7000025452, titulo=Interoperabilidad 4625452)
```

13. Patrones de diseño

Patrón	Dónde	Qué resuelve
MVC	Servidor: `model` / `servicio` / `controller`. Clientes: modelos, servicios GraphQL y pantallas.	Separar los datos, la lógica y la interfaz.
Builder	Lombok `@Builder` en `Libro` y `Ejemplar`.	Construir objetos de muchos atributos (algunos opcionales) de forma legible.
Observer	Servidor: `ObservadorColecciones` escucha los eventos de Spring Data (guardado/borrado) y aumenta la revisión en Mongo. Web: `observer/Observable.ts`. App: `observador/listado_observable.dart`.	Las pantallas de Listar y el Inicio se actualizan solas cuando cambian los datos, incluso si los cambió el otro cliente.
Singleton	`ClienteGraphQL` en la web (`getInstancia()`) y en la app (constructor de fábrica).	Una sola puerta de comunicación con el servidor por cliente.

Cómo funciona el Observer entre clientes

1. Cualquier guardado o borrado en `libros` / `ejemplares` dispara un evento de Spring Data.
2. `ObservadorColecciones` lo recibe y hace `$inc` en la colección `revisiones` (p. ej. `{ _id: "libros", valor: 15 }`).
3. Cada cliente con una pantalla de Listar abierta consulta `revisionLibros` cada 15 s; si el número cambió, vuelve a pedir la página con los mismos filtros.

14. Pruebas y resultados

Proyecto	Qué se prueba	Resultado
Servidor (`mvnw verify`, base `libreria_test`)	Servicios, reglas, validaciones, consultas propias, GraphQL completo, HTTP/CORS/X-Cliente, estructura de Mongo (índices y ObjectId), concurrencia, 3 tareas (incluida una que falla y otra que se demora) y carga.	**74/74**
Web (Vitest)	Cada pantalla: validaciones, envío, mensajes del servidor, paginación, filtros, Observer.	**84/84**
Web E2E (Playwright)	Todos los casos de uso de los dos objetos contra el servidor real con MongoDB, en escritorio y celular, con capturas.	**32/32**
App (flutter test)	Modelos, servicios, Observer y flujos de cada pantalla con un servidor simulado.	**46/46**
App integración + celular real	CRUD contra el servidor real; recorrido completo en el Galaxy S24 Ultra con 35 capturas.	**OK**
CI (GitHub Actions)	Los tres proyectos en cada push (servidor con un MongoDB 8.2 en el workflow).	**verde**

Concurrencia

- 20 hilos crean a la vez el mismo ISBN → **1 éxito y 19 "Ya existe un libro con el ISBN…"**; en Mongo queda 1 documento.
- 2 hilos actualizan con la misma versión → **1 éxito y 1 "Otro usuario modificó este libro"**.
- 2 préstamos simultáneos del mismo ejemplar → 1 conflicto; el contador queda en +1 (no se pierde ni se duplica).

Carga (1.000 libros y 5.000 ejemplares, promedio de 5 ejecuciones)

Consulta	ms
Generar 1.000 libros + 5.000 ejemplares	415
Listar libros, página 1 de 50 / página 50 de 50	7 / 7
Libros por precio 50.000–100.000 y FICCION	6
`ejemplaresConLibro` página 1 / página 250 / filtro PRESTADO	6 / 9 / 6
`resumenInventario` (3 tareas)	6

15. Guion para presentar en clase (≈ 10 min)

#	Qué mostrar	Qué decir
1	Inicio en la web y en el celular	Arquitectura: dos clientes en TypeScript y Dart, servidor Java con GraphQL, MongoDB con Spring Data. Las cifras las calculan 3 tareas en paralelo (se ven los hilos).
2	Libros → Adicionar (web), dejando un campo mal	Validación en el cliente; luego un ISBN repetido: lo rechaza el servidor (índice único).
3	Ejemplares → Adicionar (app)	El libro se elige por ISBN; el servidor guarda el ObjectId del libro como llave foránea.
4	Ejemplares → Listar (web)	Grilla con todos los atributos + título y autor del libro: consulta propia con `$lookup`. Filtro por estado, paginación. Mostrar que el ejemplar creado desde la app ya aparece (interoperabilidad).
5	Libros → Listar con precio 40.000–60.000 y un género	Segunda consulta propia (Criteria). El filtro lo hace MongoDB.
6	Consultar: libro por título “maria”; ejemplar por ISBN + “Estante A1”	Dos criterios por objeto; título sin tildes ni mayúsculas.
7	Libros → Detalle 978-0307474728	Consulta maestro + detalle con conteos por estado.
8	Ejemplares → Actualizar EJ-003 a Prestado (app)	Se busca primero y se ve toda la información; el servidor suma el préstamo (+1).
9	Actualizar el mismo libro en web y app; guardar en ambos	Concurrencia optimista con `@Version`: el segundo recibe “Otro usuario modificó este libro”.
10	Eliminar 978-0307474728 y luego Delirio	Ficha completa antes de eliminar; regla: no se borra un libro con ejemplares; Delirio sí se borra.
11	Ventana del servidor	Logs info/warn/error y la línea “App móvil / Cliente web” de cada operación.
12	Ayuda → Acerca de; GitHub	Integrantes, versión 3.0.0; repositorio con CI en verde.

> Consejo: antes de la clase, abrir la demo una vez (la primera vez compila e instala) y dejar abiertas las pantallas que se van a mostrar.

16. Preguntas probables del profesor y respuestas

¿Dónde está la lista? (está prohibida)

No hay. Los repositorios en memoria del segundo proyecto se borraron; todo pasa por `MongoRepository` / `MongoTemplate`. Las `List` que aparecen en el código son solo resultados de una consulta (lo que devuelve Mongo para una página), nunca el almacenamiento.

¿Cómo se relacionan las dos colecciones si Mongo no es relacional?

Por referencia: cada ejemplar guarda en `libroId` el `ObjectId` del `_id` de su libro. Para mostrar datos de los dos se usa `$lookup` (Listar Ejemplares) o dos consultas (Detalle). La integridad (que el libro exista y que no se borre con ejemplares) la cuida el servicio.

¿Qué es el framework?

Spring Data MongoDB: mapea las clases con `@Document`, genera las consultas derivadas por nombre (`findByIsbn`), maneja la versión (`@Version`), crea los índices (`@Indexed`) y traduce errores (por ejemplo, `DuplicateKeyException`).

¿Cuáles son las consultas creadas por ustedes?

`librosPorPrecioYGenero` (Criteria), `ejemplaresConLibro` (agregación con `$lookup`) y `detalleLibro` (maestro + detalle). Además dos búsquedas con *collation* (título y ubicación sin tildes).

¿Qué pasa si dos personas crean el mismo ISBN al mismo tiempo?

Las dos pasan la verificación previa, pero el índice único de MongoDB solo deja guardar una; la otra recibe "Ya existe…". Está probado con 20 hilos.

¿Y si dos personas editan el mismo libro?

`@Version`: cada cliente envía la versión que leyó; MongoDB solo guarda si sigue siendo la misma. El segundo recibe "Otro usuario modificó este libro; vuelva a buscarlo".

¿Dónde están las 3 tareas y cómo se ve que son concurrentes?

`tareas/ServicioResumenInventario`. En la consola cada tarea registra su inicio y fin con el nombre del hilo (`tarea-1`, `tarea-2`, `tarea-3`), y el Inicio muestra en qué hilo corrió cada una.

¿Por qué el número de préstamos no se puede editar?

Es una regla de negocio del servidor: se suma 1 cada vez que el ejemplar pasa a PRESTADO. Así ningún cliente lo puede alterar.

¿Dónde están las validaciones?

En el servidor (Bean Validation en `LibroInput` / `EjemplarInput` y la anotación propia `@IsbnValido`) y repetidas en los clientes solo para avisar antes de enviar (`validaciones.ts`, `validaciones.dart`).

¿Cómo sabe el servidor qué cliente hizo cada operación?

Cada cliente envía el encabezado `X-Cliente` (web o movil) y `RegistroOperaciones` escribe una línea por operación.

17. Mapa de archivos (dónde está cada cosa)

```
TercerPrototipo-Libreria/
├── iniciar-demo.cmd                      Doble clic: MongoDB + servidor + web + app
├── README.md                             Arquitectura, diagramas, cómo correr, rúbrica
├── .github/workflows/ci.yml              Integración continua (con MongoDB)
├── scripts/                              iniciar-demo.ps1, verificar-mongo.cmd, capturas del celular
├── servidor-libreria-mongo/              Java · Spring Boot
│   ├── src/main/java/com/libreria/servidor/
│   │   ├── model/        Libro, Ejemplar, enums, tipos de respuesta
│   │   ├── dto/          LibroInput, EjemplarInput, @IsbnValido
│   │   ├── repository/   LibroRepository(+CustomImpl: precio y género),
│   │   │                 EjemplarRepository(+CustomImpl: $lookup, conteos, suma)
│   │   ├── servicio/     ServicioLibro, ServicioEjemplar, ServicioRevisiones,
│   │   │                 ObservadorColecciones, GeneradorDatosCarga
│   │   ├── controller/   LibroController, EjemplarController, ResumenController
│   │   ├── config/       MongoConfig, EjecutorTareasConfig, RegistroOperaciones, DatosIniciales…
│   │   ├── excepciones/  LibreriaException, ManejadorErrores
│   │   └── tareas/       ServicioResumenInventario (3 tareas)
│   ├── src/main/resources/  application.properties, logback-spring.xml, graphql/schema.graphqls
│   └── src/test/java/…      74 pruebas (ConcurrenciaTest, CargaTest, GraphQlApiTest…)
├── cliente-web-react/src/
│   ├── paginas/libro/     ListarLibros, ConsultarLibro, DetalleLibro, AgregarLibro, ActualizarLibro,
EliminarLibro
│   ├── paginas/ejemplar/  ListarEjemplares, ConsultarEjemplar, AgregarEjemplar, ActualizarEjemplar,
EliminarEjemplar
│   ├── graphql/           clienteGraphQL (Singleton), servicioLibro, servicioEjemplar, servicioResumen
│   ├── observer/          Observable (Observer)
│   └── e2e/ (fuera de src) pruebas Playwright contra el servidor real
└── cliente-movil-flutter/lib/
    ├── pantallas/libro/, pantallas/ejemplar/   una pantalla por caso de uso
    ├── servicios/         cliente_graphql (Singleton), servicio_libro, servicio_ejemplar
    ├── observador/        listado_observable (Observer)
    └── modelos/, utilidades/
```

> Repositorio: **https://github.com/Milan32555/tercer-prototipo-libreria** · Capturas: `D:\01-Desarrollo\Capturas-TercerPrototipo-Libreria` (web y movil).